

Some games on Ludio allow spectators to join mid-game and become players, and/or players to leave mid-game and become spectators. These features matter because it means you can seamlessly accommodate users leaving and joining games without having to stop and restart the game you're currently playing.

Typically games that are played in rounds are good candidates for this. Social deduction games almost never allow this because they feature player elimination and/or you can't mess with the information people know about other players in the game.

To enable spectators to become players:

- Set "allowSpectatorBecomePlayer" to true in gameInitOptions
- Fill out the "turnSpectatorToPlayerActions" section which is a list of actions
 - In this section you automatically get access to a "waitingSpectator" variable in the cache, which is the player id of the new player. You can use that variable directly or copy its value to a new variable in the cache, typically "newPlayer"
- Set "turnSpectatorsToPlayers" to true at the outer-most level of an action group in the main gameLoop - the actions in turnSpectatorToPlayerActions will be run right before executing the actions in that action group

To enable players to become spectators:

- Set "allowPlayerBecomeSpectator" to true in gameInitOptions
- Fill out the "turnPlayerToSpectatorActions" section which is a list of actions
- Set "turnPlayersToSpectators" to true at the outer-most level of an action group in the main gameLoop - the actions in turnPlayerToSpectatorActions will be run right before executing the actions in that action group

The complexity arises in filling out turnSpectatorToPlayerActions and turnPlayerToSpectatorActions correctly.

Some considerations:

- You will have to recompute and save the "players" and "numPlayers" variables to cache. There will likely be other player-specific cached variables that need to get updated.
- When spectators become players:
 - You have to set their role.
 - If the game involves scoring, we typically set the score of the new player to the lowest current score of anyone in the game (if high scores are better) or the highest current score of anyone in the game (if lower scores are better). The lowest current score of anyone in the game has to be computed in the game loop and saved, because attempting to compute it in turnSpectatorToPlayerActions will automatically pick up the new player's score as 0 (which will often be the lowest score, but not the score we want).
- In a card game, you have to carefully manage the cards of the player who is leaving or joining, as well as any player-related information in the central widget. If a new player is joining, you may have to deal them one-off cards or modify the deck used in the entire

game (e.g. trick-taking card games remove cards from the game based on the player count). If a player is leaving, you may have to recall the cards in their hand back to specific decks. You may also have to recreate the central widget to account for the new/missing player.

The Emeralds game json is a great example for how to handle players becoming spectators and vice versa in a card game.